

Chapitre IV:

Les curseurs



- ▶ Chaque fois qu'une commande SELECT, INSERT, UPDATE ou DELETE est soumise pour exécution, une zone de travail de l'environnement utilisateur lui est allouée (dans la mémoire PGA Program Global Area).
- ▶ Cette zone mémoire est appelée CURSEUR (CONTEXT AREA). Elle contient des informations permettant le traitement de l'ordre SQL.
- ▶ Le curseur permet de faire un à un les enregistrements (lignes de tables) ramenés par l'instruction SQL en question.



- PL/SQL distingue les **curseurs implicites** et les **curseurs explicites**

▼ curseurs implicites

- Lorsqu'un utilisateur lance une commande SQL, Oracle génère un curseur pour le traitement de cette commande. Ce curseur créé et géré par Oracle est dit curseur implicite.
- Ils ne sont pas préalablement définis dans la section DECLARE, et existent à l'endroit où ils doivent être exécutés dans le bloc.
- Un curseur implicite est une commande parmi :
 - SELECT qui doit ramener exactement une seule ligne.
 - INSERT, UPDATE ou DELETE sans aucune contrainte sur le nombre de n-uplets affectés.



- PL/SQL distingue les **curseurs implicites** et les **curseurs explicites**

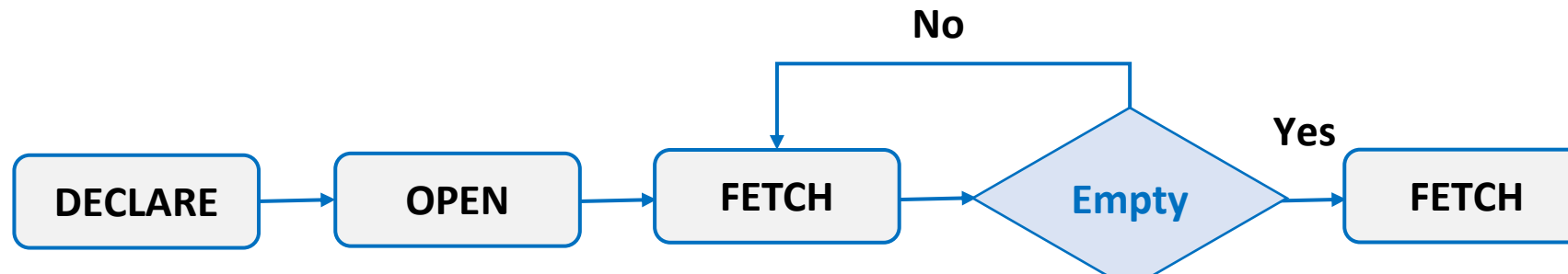
▼ curseurs explicites

- Lorsqu'un utilisateur souhaite gérer une commande SQL au sein de son code PL/SQL, il peut créer explicitement un curseur pour traiter une à une les lignes ramenées par la commande.
- Un curseur explicite doit être explicitement :
 - Déclaré dans la section DECLARE.
 - Géré par le développeur dans la section exécutable.

La gestion d'un tel curseur consiste à exécuter les opérations : ouverture du curseur, lecture et traitement de ses lignes, et enfin sa fermeture.



- PL/SQL distingue les curseurs implicites et les curseurs explicites



```
DECLARE  
nom employees.last_name%TYPE;  
Salaire employees.salary%TYPE;  
CURSOR C1 IS SELECT last_name, NVL(salary,0)  
FROM employees;  
BEGIN  
OPEN C1;  
LOOP
```

```
FETCH C1 INTO nom,salaire;  
EXIT WHEN c1%NOTFOUND;  
DBMS_OUTPUT,PUT_LINE(nom || ' gagne' ||  
salaire || 'dollars');  
END LOOP;  
DBMS_OUTPUT,PUT_LINE(C1%ROWCOUNT);  
CLOSE C1;  
END;  
/
```



Etapes de manipulation

♣ Déclaration

```
DECLARE  
CURSOR nomCurseur [(nomparam type ,[ nomparam type, )]  
IS commande SELECT;  
BEGIN
```

```
DECLARE  
nom employees.last_name%TYPE;  
Salaire employees.salary%TYPE;  
CURSOR C1 IS SELECT last_name, NVL(salary,0)  
FROM employees;  
BEGIN  
OPEN C1;  
LOOP
```



Etapes de manipulation

♣ Ouverture

```
OPEN nomCurseur [(nomparm1 [, nomparm2,]] ...)
```

```
DECLARE  
nom employees.last_name%TYPE;  
Salaire employees.salary%TYPE;  
CURSOR C1 IS SELECT last_name, NVL(salary,0)  
FROM employees;  
BEGIN  
OPEN C1;  
LOOP
```



Etapes de manipulation

♣ Ouverture

```
OPEN nomCurseur [(nomparm1 [, nomparm2,]) ...]
```

L'étape d'ouverture permet :

- L'allocation mémoire
- L'analyse sémantique et syntaxique de l'ordre SQL
- Le positionnement de verrous éventuels (SELECT ... FOR UPDATE)

```
DECLARE
```

```
nom employees.last_name%TYPE;
```

```
Salaire employees.salary%TYPE;
```

```
CURSOR C1 IS SELECT last_name, NVL(salary,0)  
FROM employees;
```

```
BEGIN
```

```
OPEN C1;
```

```
LOOP
```




Etapes de manipulation

♣ Traitement des lignes

```
FETCH nomCurseur INTO  
{nomvariable[,nomvariable]... | nomrecord };
```

```
FETCH C1 INTO nom,salaire;  
EXIT WHEN c1%NOTFOUND;
```

- Il faut traiter les lignes **une par une** et **déclarer les variables réceptrices dans la partie DECLARE**.
INE(nom||' gagne' ||
- La commande FETCH...INTO, utilisée à l'intérieure d'une boucle, permet de lire séquentiellement les lignes du curseur dans l'ordre, de la première à la dernière, **sans possibilité de retour ni de saut**.
INE(C1%ROWCOUNT);
- A Chaque FETCH, le contenu de la **ligne courante est affecté** dans les variables de la clause INTO.



Etapes de manipulation

♣ Fermeture

```
CLOSE nomCurseur;
```

L'étape de fermeture permet la libération de l'espace mémoire.

```
FETCH C1 INTO nom,salaire;  
EXIT WHEN c1%NOTFOUND;  
DBMS_OUTPUT,PUT_LINE(nom || ' gagne' ||  
salaire || 'dollars');  
END LOOP;  
DBMS_OUTPUT,PUT_LINE(C1%ROWCOUNT);  
CLOSE C1;  
END;  
/
```



► Les attributs d'un curseur fournissent des informations quant à l'exécution de l'ordre.

► Ces informations sont conservées par PL/SQL après l'exécution du curseur (explicite ou implicite).

► Ces attributs permettent de tester directement le résultat de l'exécution de l'ordre SQL.

Attribut	Type	Description
%ISOPEN	Booléen	Prend la valeur TRUE si le curseur est ouvert
%NOTFOUND	Booléen	Prend la valeur TRUE si la dernière extraction (fetch) ne renvoie pas de ligne
%FOUND	Booléen	Prend la valeur TRUE si la dernière extraction renvoie une ligne ; complément de %NOTFOUND
%ROWCOUNT	Nombre	Prend la valeur correspondant au nombre total de lignes renvoyées jusqu'à présent



- ▶ Le curseur IMPLICITE est déclaré directement dans une structure FOR IN
- ▶ Cette structure permet de:
 - Eviter de déclarer le curseur dans la partie DECLARE
 - Avoir une ouverture automatique du curseur (OPEN)
 - Faire un FETCH automatique
 - Avoir une condition de sortie automatique
 - Avoir Fermeture automatique du curseur

```
For variable IN nom_curseur LOOP  
--Instructions  
END LOOP;
```



Les attributs du curseur

- Ecrire un bloc anonyme PL/SQL qui affiche la liste des employés soit leur nom, leur id et leur salaire trié par salaire descendant. Utilisez un curseur implicite.

```
DECLARE
i number :=0;
BEGIN
FOR c1 IN (SELECT last_name, employee_id, salary FROM employees
          ORDER BY salary DESC)
LOOP
i:=i+1;
dbms_output.put_line('Employee ' || c1.last_name || ' (' || c1.employee_id
|| ') touche ' || c1.salary);

EXIT WHEN i=5;
END LOOP;
END; /
```



Nom Attribut	Type	Valeur	Explication
nomCurseur%ISOPEN	BOOLEAN	TRUE	Si le curseur est ouvert
		FALSE	Si le curseur est fermé (ou non encore ouvert)
nomCurseur%FOUND	BOOLEAN	TRUE	Si le dernier FETCH ramène une ligne
		FALSE	Si le dernier FETCH ne trouve plus une ligne à ramener
		NULL	Avant le premier FETCH (curseur ouvert)



nomCurseur%NOTFOUND	BOOLEAN	TRUE	Si le dernier FETCH ne trouve plus une ligne à ramener
		FALSE	Si le dernier FETCH ramène une ligne
		NULL	Avant le premier FETCH (curseur ouvert)
nomCurseur%ROWCOUNT	NUMBER	Entier	Compteur qui s'incrémente après chaque ligne lue (par FETCH)
		Zéro	Après OPEN et avant le premier FETCH
nomCurseur%ROWTYPE			permet la déclaration implicite d'une structure dont les éléments sont d'un type identique aux colonnes ramenées par le curseur.



Soit la table suivante :

CLIENT (NCL, NOM, VILLE, CA, CATEGORIE)

1. On veut mettre à jour le champ CATEGORIE pour chaque client de la ville de 'Tunis' selon les formalités suivantes (utiliser un curseur explicite) :

- Si $CA > 100000$ Dinars, alors CATEGORIE est : 'CLIENT POTENTIEL'
- Si $CA \leq 100000$ et $CA > 20000$ Dinars, alors CATEGORIE est : 'CLIENT MOYEN'
- Si $CA \leq 20000$ Dinars, alors CATEGORIE est : 'CLIENT PASSAGER'

2. On veut afficher les 10 premiers clients potentiels de la ville de 'Tunis', ordonnés selon leurs chiffres d'affaires (CA). (utiliser un curseur explicite)



1. Utilisation des attributs : %found et %isopen
2. Utilisation des attributs : %rowtype et %notfound
3. Utilisation des attributs : %rowtype, %notfound et %rowcount



► L'objectif est de fournir au programmeur une structure simple et efficace pour utiliser les structures répétitives et les curseurs.

```
DECLARE
    Cursor nomCurseur is ordre_select;
    Nomrecord nomCurseur%ROWTYPE;
BEGIN
    Open nomcurseur;
LOOP
    FETCH nomcurseur into nomrecord;
    Exit when nomcurseur%NOTFOUND
    -- Traitement
END LOOP;
    Close nomCurseur;
END;
```



► On remarque qu'avec la boucle For, on ait privé d'ouvrir le curseur, de le fermer et de lancer l'ordre fetch. Ça suppose également la déclaration implicite d'une variable de type structure dans laquelle sera sauvegardé le résultat renvoyé par le curseur.

```
DECLARE
    Cursor nomCurseur is ordre_select;
BEGIN
    For NomRecord In nomCurseur LOOP
        --traitement
    END LOOP;
END;
```



- ▶ Les curseurs paramétrés permettent d'utiliser des variables dans le curseur. Principalement dans la clause Where.
- ▶ Il Un curseur paramétré peut servir plusieurs fois avec des valeurs des paramètres différentes.
- ▶ Il faut pour cela spécifier les noms et les types des paramètres dans la déclaration du curseur.
- ▶ On doit fermer le curseur entre chaque utilisation de paramètres différents (sauf si on utilise « for » qui ferme automatiquement le curseur

```
CURSOR nomCurseur (param1 type, param2 type, ...)  
Is Ordre_Select;
```



Soit la BD suivante :

DEPARTEMENT(NumDep, NomDep)

EMPLOYEE (NumEmp, NomEmp, Fonction, Salaire, Commission, #NumDep)

**Q1: Augmenter de 10% les salaires des employés travaillant dans le département Numéro 20.
(Utiliser un curseur paramétré)**



Soit la BD suivante :

DEPARTEMENT(NumDep, NomDep)

EMPLOYEE (NumEmp, NomEmp, Fonction, Salaire, Commission, #NumDep)

Q2: Afficher la liste des employés par département triés selon leurs salaires



- Les commandes SQL (UPDATE et DELETE) peuvent utiliser la clause WHERE CURRENT OF nomCurseur pour référencer la ligne courante d'un curseur explicite ramenée par l'ordre FETCH.
- Ceci permet d'établir une correspondance entre la ligne courante du curseur et son n-uplet source à modifier (UPDATE) ou à supprimer (DELETE) dans la table de définition du curseur.
- L'utilisation de WHERE CURRENT OF nomCurseur exige que le curseur ait été défini avec la clause FOR UPDATE OF nomColonne (positionnement d'un verrou d'intention).